

Symulacja i analiza procesów w sieciach złożonych

Maciej Polak Grzegorz Śnieżko Karol Klimek

13 czerwca 2026

Spis treści

- 1 Wstęp
- 2 Opis problemu
- 3 Analiza
- 4 Fragmenty kodu

Wprowadzenie do teorii sieci

Sieci złożone, nazywane również grafami, swój początek datują na rok 1736. To wtedy **Leonhard Euler** opisał matematyczną niemożliwość przejścia tylko jeden raz przez siedem mostów znajdujących się w ówczesnym Królewcu (dzisiejszym Kaliningradzie). Analiza Eulera polegała na przedstawieniu obszarów lądu jako punktów na mapie oraz połączeniu ich liniami reprezentującymi trasy (mosty). Praca ta położyła fundamenty pod dziedzinę, którą dzisiaj znamy jako teorię grafów i sieci złożonych.

Charakterystyka i zakres badania

- **Cel:** Analiza przepływu pasażerów oraz przepustowości sieci transportowej.
- **Dane:** Statystyki ULC z lipca 2009 roku, przerobione na potrzeby analizy.
- **Narzędzia:** *SimPy* (symulacja procesów), *NetworkX* oraz *igraph*.

Analizowana sieć obejmuje 10 kluczowych portów lotniczych:

- | | |
|-----------------------------------|---------------------------------|
| • WAW: Warszawa Okęcie | • POZ: Poznań-Ławica |
| • KRK: Kraków J. Paweł II | • RZE: Rzeszów-Jasionka |
| • KTW: Katowice-Pyrzowice | • LCJ: Łódź (Reymonta) |
| • GDN: Gdańsk im. Wałęsy | • SZZ: Szczecin-Goleniów |
| • WRO: Wrocław Strachowice | • BZG: Bydgoszcz |

Narzędzia badawcze: NetworkX i igraph

Do modelowania i analizy topologii sieci wykorzystano dwie wiodące biblioteki analityczne języka Python:

NetworkX

- Elastyczne środowisko do tworzenia i modyfikacji struktur grafowych.
- W naszym projekcie posłużyło do budowy modelu tras lotniczych, przechowywania atrybutów węzłów (np. współrzędnych) oraz wyliczania wag czasowych krawędzi.

igraph

- Biblioteka o niezwykle wysokiej wydajności (dzięki rdzeniowi napisanemu w języku C), zoptymalizowana do analizy złożonych sieci.
- Wykorzystana do błyskawicznych, zaawansowanych obliczeń strukturalnych.

Narzędzia badawcze: SimPy

Do analizy dynamiki systemu i zatorów posłużyła biblioteka **SimPy** — zorientowany na procesy framework symulacji zdarzeń dyskretnych (DES).

Opiera się ona na asynchronicznych generatorach języka Python (z wykorzystaniem instrukcji `yield`). Kluczowe mechanizmy użyte w modelu to:

- **Środowisko (*Environment*)**: Centralny zegar, który zarządza czasem w minutach i dysponuje zdarzeniami (lotami).
- **Procesy (*Processes*)**: Każdy generowany lot w sieci funkcjonuje jako niezależny, równoległy proces.
- **Zasoby (*Resources*)**: Limitowane stanowiska (bramki) na poszczególnych lotniskach. Wyczerpanie puli bramek wymusza oczekiwanie procesów, co naturalnie generuje opóźnienia kaskadowe w symulacji.

Modelowanie dynamiki ruchu (SimPy)

Aby zbadać przepustowość sieci w warunkach zbliżonych do rzeczywistych, przeprowadzono dyskretną symulację zdarzeń.

Parametry modelu:

- **Czas symulacji:** 1 doba (1440 minut).
- **Generowanie ruchu:** Loty dla każdej trasy pojawiają się w interwałach czasowych zgodnych z rozkładem Poissona.
- **Ograniczenia zasobów:** Każde lotnisko posiada skończoną liczbę stanowisk obsługi (bramek).
- **Proces lotu:** Żądanie bramki w porcie A $\rightarrow$ zajęcie bramki (boarding) $\rightarrow$ lot (z losowym odchyleniem czasu) $\rightarrow$ żądanie bramki w porcie B $\rightarrow$ zwolnienie zasobów.

Analiza topologiczna: Struktura Hub-and-Spoke

Analiza topologii ukazuje wyraźną dominację węzła centralnego, tworząc strukturę *Hub-and-Spoke* (piasta i szprychy). Jest to klasyczny przykład **sieci bezskalowej** — wysoce odpornej na losowe awarie portów regionalnych, lecz skrajnie podatnej na paraliż głównego huba.

Kluczowe metryki centralności:

- **PageRank**: Lotnisko Chopina (WAW) osiąga najwyższą wartość, co wynika z obsługi największej liczby relacji i operacji lotniczych.
- **Pośrednictwo (Betweenness Centrality)**: WAW pełni rolę głównego punktu przesiadkowego. Awaria tego węzła najbardziej zakłóciłaby przepływ w całej sieci.

Odwzorowanie struktury sieci w kodzie

```
# Definicja wezlow i krawedzi (fragment)
AIRPORTS = {
    "WAW": {"name": "Warszawa", "gates": 12},
    "KRK": {"name": "Krakow", "gates": 6},
    "KTW": {"name": "Katowice", "gates": 6}
}

ROUTES = [
    ("WAW", "KRK", 252, 10), ("WAW", "GDN", 340, 10),
    ("KRK", "GDN", 550, 4)
]

# Budowa grafu i wag czasowych
G_net = nx.Graph()
for s, d, dist, f in ROUTES:
    travel_time = round((dist / 700) * 60) # km / kmh
    G_net.add_edge(s, d, travel_min=travel_time, base_freq=f)
```

Proces lotu w środowisku SimPy

```
def flight_process(env, f_id, src, dst, airports_res):  
    # 1. Oczekiwanie na otwarcie lotniska startowego  
    while not AIRPORT_STATUS[src]['is_open']:  
        yield env.timeout(3)  
  
    # 2. Zajęcie bramki na lotnisku startowym i boarding  
    with airports_res[src].request() as req:  
        yield req  
        yield env.timeout(random.uniform(10, 20))  
  
    # 3. Przelot (czas pobrany z topologii grafu)  
    yield env.timeout(G_net[src][dst]['travel_min'])  
  
    # 4. Przekierowanie w przypadku zamknięcia lotniska docelowego  
    if not AIRPORT_STATUS[dst]['is_open']:  
        dst, extra_time = find_backup(dst)  
        yield env.timeout(extra_time)  
  
    # 5. Ladowanie i zwolnienie zasobow w porcie docelowym  
    with airports_res[dst].request() as req:  
        yield req  
        yield env.timeout(random.uniform(10, 15))
```

Generowanie ruchu pasażerskiego

```
def route_generator(env, src, dst, airports_res):  
    # Wyznaczenie sredniego odstepu miedzy lotami  
    mean_interval = SIM_DURATION_MIN / (G_net[src][dst]['  
    base_freq'] * 3)  
  
    idx = 1  
    while True:  
        # Losowanie czasu do nastepnego lotu (Rozklad  
        Poissona)  
        yield env.timeout(random.expovariate(1.0 /  
        mean_interval))  
  
        # Uruchomienie niezaleznego procesu dla nowego lotu  
        env.process(flight_process(env, f"{src}{idx}", src,  
        dst, airports_res))  
        idx += 1
```

Symulacja anomalii i odporności sieci

```
# Konfiguracja awarii: (Lotnisko, minuta startu, czas
    trwania)
ANOMALIES_CONFIG = [
    ("KRK", 550, 240),
    ("LCJ", 200, 150),
    ("WAW", 300, 240) # Blokada glownego huba na 4 godziny!
]

def anomaly_trigger(env, target, start, duration):
    yield env.timeout(start)
    AIRPORT_STATUS[target]['is_open'] = False

    yield env.timeout(duration)
    AIRPORT_STATUS[target]['is_open'] = True
```

Wnioski z symulacji: Zamknięcie węzła WAW w 300. minucie wywołuje kaskadowe opóźnienia w całej Polsce, ponieważ samoloty nie mogą zwolnić bramek w portach regionalnych.